

(final version will be on the ACM website)

# Mining High Utility Itemsets with Hill Climbing and Simulated Annealing

M. SAQIB NAWAZ, School of Humanities and Social Sciences, Harbin Institute of Technology (Shenzhen), China

PHILIPPE FOURNIER-VIGER\*, School of Humanities and Social Sciences, Harbin Institute of Technology (Shenzhen), China

UNIL YUN, Department of Computer Engineering, Sejong University, Korea

YOUXI WU, Department of Computer Science and Engineering, Hebei University of Technology, China

WEI SONG, School of Information Science and Technology, North China University of Technology, China

High utility itemset mining (HUIM) is the task of finding all items set, purchased together, that generate a high profit in a transaction database. In the past, several algorithms have been developed to mine HUIs. However, most of them cannot properly handle the exponential search space while finding HUIs when the size of the database and total number of items increases. Recently, evolutionary and heuristic algorithms were designed to mine HUIs, which provided a considerable performance improvement. However, they can still have a long runtime and some may miss many HUIs. To address this problem, this paper proposes two algorithms for HUIM based on Hill Climbing (HUIM-HC) and Simulated Annealing (HUIM-SA). Both algorithms transform the input database into a bitmap for efficient utility computation and for search space pruning. To improve population diversity, HUIs discovered by evolution are used as target values for the next population instead of keeping the current optimal values in the next population. Through experiments on real-life datasets, it was found that the proposed algorithms are faster than state-of-the-art heuristic and evolutionary HUIM algorithms, that HUIM-SA discovers similar HUIs, and that HUIM-SA evolves linearly with the number of iterations.

CCS Concepts: • **Information systems** → **Data mining**; • **Theory of computation** → **Simulated annealing**; *Evolutionary algorithms*.

Additional Key Words and Phrases: Hill climbing, Simulated annealing, High utility itemsets, Bitmap, Neighbor

## ACM Reference Format:

M. Saqib Nawaz, Philippe Fournier-Viger, Unil Yun, Youxi Wu, and Wei Song. 2021. Mining High Utility Itemsets with Hill Climbing and Simulated Annealing. *ACM Trans. Manag. Inform. Syst.* 0, 0, Article 0 (2021), 23 pages. <https://doi.org/XXXXX>

\*corresponding author

Authors' addresses: M. Saqib Nawaz, [msaqibnawaz@hit.edu.cn](mailto:msaqibnawaz@hit.edu.cn), School of Humanities and Social Sciences, Harbin Institute of Technology (Shenzhen), Shenzhen, China; Philippe Fournier-Viger, School of Humanities and Social Sciences, Harbin Institute of Technology (Shenzhen), Shenzhen, China, [philfv8@yahoo.com](mailto:philfv8@yahoo.com); Unil Yun, Department of Computer Engineering, Sejong University, Seoul, Korea, [yunei@sejong.ac.kr](mailto:yunei@sejong.ac.kr); Youxi Wu, Department of Computer Science and Engineering, Hebei University of Technology, Tianjin, China, [wuc567@163.com](mailto:wuc567@163.com); Wei Song, School of Information Science and Technology, North China University of Technology, Beijing, China, [songwei@ncut.edu.cn](mailto:songwei@ncut.edu.cn).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2021 Association for Computing Machinery.

2158-656X/2021/0-ART0 \$15.00

<https://doi.org/XXXXX>

## 1 INTRODUCTION

Pattern mining [1] algorithms are used in data mining to identify not only unexpected but useful and interesting patterns in large databases. This process is typically used to explore the data to discover interesting relationships between values. The patterns that are discovered can also help to take decisions or make future predictions. To find patterns, users typically need to set constraints on what is an interesting pattern. Then, an algorithm can extract all the patterns meeting these requirements. In the past, various algorithms have been proposed and developed to find all kinds of patterns in different types of data. Some of the most popular pattern mining problems are Frequent Itemset Mining (FIM) [2] and Association Rule Mining (ARM) [3]. In FIM, the main aim is to find sets of items (symbols) that have a support (number of occurrences) that is greater or equal to a minimum support threshold that is set by the user. The goal of ARM is similar to FIM but patterns are represented as rules instead of sets, and not only the support is measured but also the confidence (an estimation of the conditional probability) that a rule is followed. ARM and FIM can be applied in various domains such as to find frequent purchases made by customers in a store or study frequently co-occurring words in a text. However, an underlying assumption of these tasks is that the frequency is an appropriate measure for selecting interesting patterns. But this is not always the case. For instance, some patterns of purchases made by customers in a store may be very frequent but they may generate a very low profit. Thus, such patterns are uninteresting and unimportant for decision-makers.

To provide a more general definition of the importance of a pattern, the problem of FIM was generalized as that of high utility itemset mining (HUIM) [4]. HUIM takes as input a quantitative transaction database that contains transactions (records). Each transaction in the database contains a set of items with quantities, and each item has a weight that represents the relative importance of that item. The goal of HUIM is then to find high utility itemsets (HUIs), that is patterns that have a utility value that is no less than a user-specified minimum utility threshold. In the context of mining patterns in customer data, the utility of an itemset can represent the total amount of profit yield by the purchase of its items. HUIM can be viewed as more useful than FIM in that scenario as it allows finding the most profitable sets of items purchased by customers rather than the most frequent ones. Besides analyzing shopping data, HUIM also has many other applications such as market basket analysis [5], analyzing click-stream data where the utility can for example represent the time spent by visitors on a website [4], regulation of gene expression [6], analyzing data obtained from mobile commerce environments [7] and finding recent high-utility patterns in temporal data [8].

In literature, several algorithms can be found that can efficiently discover all high utility itemsets in a quantitative database [9–14]. These algorithms for HUIs generally have the same input and output. The differences between them lie in the different data structures and strategies used in these algorithms for searching and finding HUIs. More specifically, they differ in:

- whether a breadth-first search or depth-first is used,
- the database representation type (vertical or horizontal),
- how the next itemsets that need to be explored more in the search space is determined, and
- how the utility of itemsets are calculated to check whether they satisfy the minimum utility constraint or not.

However, enumerating all HUIs in a database is difficult to achieve and the performance of existing exact algorithms for HUIM degrade when the database size and the total number of distinct items in the databases increases. Moreover, HUIs are often scattered in the search space. This forces the algorithm to (must) consider many itemsets before it can discover the actual HUIs.

To deal with the performance bottleneck of traditional pattern mining algorithms, a promising approach has been to adopt approximate algorithms that can provide a good trade-off between completeness and performance [15, 16]. In particular, much attention has been given to developing evolution based and heuristic search techniques for HUIM in recent years, as these techniques have the ability to solve complex, linear as well as highly nonlinear problems [17]. They can explore large problem spaces to find a near optimal solution on the basis of fitness functions under a set of multiple constraints. Genetic algorithm (GA) was first used in [18] to mine HUIs. Two GA-based algorithms, namely  $HUPE_{UMU}$ -GARM and  $HUPE_{WUMU}$ -GARM, were proposed for HUIM. Particle Swarm Optimization (PSO) was also utilized to mine HUIs [19, 20]. *BATARM* [21] used the bat algorithm for ARM, and was later modified to develop the cooperative multi-swarm bat algorithm called *MSB-ARM* for ARM [22]. Though the existing evolutionary-based HUIM algorithms can provide a runtime improvement over traditional exact algorithms to mine all HUIs that satisfy the minimum utility threshold, they can still take a lot of time. The main reason for this is that these algorithms follow the general routines of the standard evolutionary and search algorithm. This means that optimal values of one population are kept in the next population and the search space is explored further on the basis of optimal values obtained in the previous population. This kind of strategy for searching is more suitable for problems that have less optimal values. However, for the problem of HUIM, the total number of results (possible HUIs) can be very large. As HUIs are not evenly distributed, thus searching them with the optimal values that are obtained from the previous population as targets can make the algorithm to miss some results within a certain number of iterations.

A solution to overcome this problem is to enhance the diversity of the generated population. A bio-inspired framework for HUIM was proposed by Song and Huang [23], where the initial target of the next population was probabilistically determined by applying roulette wheel selection on all the discovered HUIs, instead of choosing HUIs with high utility values from the current population. Moreover, two strategies, database representation using bitmap and promising encoding vector checking, were used to accelerate the process of HUIM. Under that framework, three algorithms were proposed based on GA, PSO, and the Bat algorithm (BA). Moreover, Artificial Bee Colony (ABC) algorithm was utilized in [24] to mine HUIs based on the developed framework [23].

To further reduce runtimes while ensuring good result quality for mining HUIs in large databases, this paper proposes two novel approximate algorithms based on Hill Climbing (HC) [25] and Simulated Annealing (SA) [26], respectively. To the best of our knowledge, these two searching and meta-heuristic techniques were not used to mine HUIs. Taking the work done in [23, 24] as starting point, we modeled the problem of HUIM from the perspective of HC and SA algorithms. The database is converted into a bitmap, which is used both for information representation and search space pruning. Instead of maintaining the HUIs with highest utility value from population to population, the strategy of selecting discovered HUIs probabilistically for the next population is used. This strategy allows to discover more HUIs in less iteration cycles. Extensive experimental results are performed on six real-life datasets to investigate and compare the performance of HUIM-HC and HUIM-SA with existing evolutionary based HUIM algorithms.

The remaining paper is organized as follows. Section 2 gives an overview of related work. Section 3 describes the problem of HUIM. Then, Section 4 presents the proposed HUIM approaches based on HC and SA. Thereafter, Section 5 presents and discusses the experiments and obtained results. Finally, the paper is concluded with some remarks in Section 6.

## 2 RELATED WORK

The problem of FIM was proposed by Agrawal and Srikant [27] to find sets of symbols (items) that appear at least some minimum number of times in a database. The occurrence frequency of a

pattern is called the support and has the nice property of being anti-monotonic, that is an itemset cannot have a superset having a higher support, and hence supersets of an infrequent itemset must also be infrequent. In FIM, the search space can be very large. Generally, if a database contains  $n$  distinct items, there are  $2^n - 1$  possible itemsets. For some applications like market basket analysis on webstores,  $n$  can be greater than 1 million. But thanks to the anti-monotonicity of the support measure, frequent itemsets are clustered next to each other in the search space, and a large part of the search space can thus be eliminated. This has led to the design of several exact algorithms that are relatively efficient for this problem [2]. However, despite that FIM has many applications, its input data format remains quite simple. It can be viewed as a table of record described using binary attributes. Hence, it cannot model well the data in several domains. Moreover, frequent patterns are not always interesting and other criteria should be considered.

The problem of HUIM was proposed to address these limitations by generalizing FIM [28] for transaction databases where each transaction has item quantities, and items have weights (e.g. to represent the unit profit of items). Then, the aim of HUIM is to find those itemsets that have a utility (importance) greater than or equal to a minimum utility threshold. Generally, the problem of HUIM is much harder than FIM because the utility function is not anti-monotonic contrarily to the support measure (and also neither monotonic). As a result, high utility itemsets can be scattered in the search space and the utility cannot be used directly for the reduction of search space. Several exact algorithms were proposed for HUIM such as Two-Phase [9], BAHUI [29], UP-Growth [11], FHM [13] and EFIM [14], and it is an active research area [4]. To effectively reduce the search space, the aforementioned exact algorithms have introduced various upper bounds on the utility of itemsets that are anti-monotonic such as the TWU upper bound [9]. However, these upper bounds can be quite loose and as a result many low utility itemsets are often evaluated to find the true HUIs, which deteriorates the performance.

Although exact FIM and HUIM algorithms guarantee providing complete results, their runtimes can be very long. Especially, when the user sets the minimum threshold too low, it is not uncommon that an algorithm can run for several hours or more, or may even have to be stopped before terminating. Moreover, the search space tends to become very large for databases with many transactions, long transactions, and/or with many distinct items. To address these issues, a promising approach has been to develop evolutionary and heuristic-based algorithm [15, 16]. The idea is to find an excellent trade-off between speedup and completeness. Some of these algorithms can in fact find most desired itemsets in a fraction of the time of an exact algorithm. Moreover, evolutionary and heuristic based algorithms typically iteratively improve the current solution and it is thus easy to stop them at any time to obtain results [16]. Thus, these algorithms can be viewed as more practical.

The first work on evolutionary and heuristic-based algorithms for pattern mining were for FIM and ARM. For example, the studies in [30–32] and [33–38] proposed GA for FIM and ARM, respectively. PSO [39, 40] and BA [21, 22] were also used for ARM. For HUIM, evolutionary and meta-heuristic-based algorithms have been used [18–20, 23, 24, 41, 42, 44]. The two GAs proposed in [18] for HUIM used the common operators (selection, crossover and mutation) iteratively to find HUIs. However, initially they cannot easily find the 1-HTWUIs as chromosomes and thus they need a huge computation for setting the appropriate chromosomes for mining valid HUIs. Additionally, setting the appropriate values for some specific parameters is a non-trivial task. The performance of HUIM-GA [18] was later improved [19] by using the OR/NOR-tree structure for pruning. A bio-inspired framework was proposed in [23] that implements GA for HUIM. In the framework, efficient strategies for database representation and a pruning process were used to accelerate the HUIs discovery process. Additionally, an improved GA was proposed [41] that used several novel strategies to efficiently mine HUIs.

Besides GA, the bio-inspired framework [23] also implemented PSO and the BA to mine HUIs in large databases. Additionally, that framework [23] was utilized in [24] to implement the ABC algorithm to solve the problem of HUIM. A recent work [43] proposed two PSO algorithms (the first is based on standard PSO and the second is based on bio-inspired framework for HUIM [23]) to solve the problem of high average-utility itemset mining (HAUIM). The work [44] used a Boolean-based Grey wolf algorithm (called BGWO-HUI) for the problem of HUIM. Moreover, a binary PSO was adopted in [20] and a binary PSO with OR/NOR-tree structure in [19] to efficiently mine HUIs. An Ant Colony System (ACS) was also used [42] to find HUIs. In the proposed HUIM-ACS, the complete solution space was mapped into the routing graph and two novel pruning strategies were used for accelerating the algorithm convergence.

Though, the above evolutionary and heuristic-based algorithms for HUIM were shown to achieve better performances than state-of-the-art exact HUIM algorithms, runtimes can still be quite long. Moreover, some of the above algorithms can miss several HUIs. The next section presents preliminaries for HUIM and then the following section presents the proposed HC and SA-based HUIM algorithms that aim at addressing the above limitations by enhancing the population diversity in each iteration.

### 3 PRELIMINARIES

In this section, the main concepts of HUIM is presented followed by a formal problem definition. Let  $I = \{i_1, i_2, \dots, i_m\}$  represents a finite set of  $m$  distinct items and  $TD = \{T_0, T_1, T_2, \dots, T_n\}$  be a transaction database. In  $TD$ , each transaction  $T_c$  is a subset of  $I$  and has a unique identifier  $c$  ( $1 \leq c \leq n$ ) called its TID. The set  $X \subseteq I$  is called an *itemset* and an itemset that contains  $k$  items is called a  $k$ -itemset. An itemset  $X$  is contained in a transaction  $T_c$  if  $X \subseteq T_c$ . Every item  $i_j$  in  $T_c$  has a positive number  $q(i_j, T_c)$ , called its *internal utility*, that represents the quantity (occurrence) of  $i_j$  in  $T_c$ . Another positive number, called the *external utility*  $p(i_j)$ , represents the unit profit value of the item  $i_j$ . A profit table  $p_{table} = \{p_1, p_2, \dots, p_m\}$  shows the profit value  $p_j$  of each item  $i_j$  in  $I$ .

For example, consider the transaction database depicted in Table 1 as the running example. Table 1 has six transactions and six distinct items (from  $a$ - $f$ ). Consider the sixth transaction ( $T_5$ ). This transaction indicates that a customer have bought 1, 4 and 2 units of some items  $b$ ,  $d$  and  $f$ , respectively. Table 2 lists the profit value (external utility) of each item. For example, it indicates that the sale of one unit of item  $a$  yield a 2\$ profit.

Table 1. A transaction database with internal utility values

| TID   | Transactions                | TU |
|-------|-----------------------------|----|
| $T_0$ | (a, 3) (c, 12) (e, 3)       | 54 |
| $T_1$ | (b, 4) (d, 2) (e, 1) (f, 5) | 47 |
| $T_2$ | (a, 3) (c, 2) (e, 1)        | 16 |
| $T_3$ | (a, 2) (d, 2) (f, 1)        | 15 |
| $T_4$ | (a, 1) (c, 5) (d, 7)        | 52 |
| $T_5$ | (b, 1) (d, 4) (f, 2)        | 29 |

The overall utility of an item  $i_j$  in a transaction  $T_c$  is defined as:

$$u(i_j, T_c) = p(i_j) \times q(i_j, T_c) \quad (1)$$

The *utility of an itemset  $X$  in a transaction  $T_c$*  is denoted as  $u(X, T_c)$  and represents the money obtained from the sale of  $X$  in that transaction. Moreover, the *overall utility of an itemset  $X$  in  $TD$*  is

Table 2. External profit values of items

| Item   | a | b | c | d | e | f |
|--------|---|---|---|---|---|---|
| profit | 2 | 7 | 3 | 5 | 4 | 1 |

denoted as  $u(X)$  and represents the total amount of money that the itemset yield for all transactions where  $X$  is purchased in the database. These two concepts are defined formally as follows:

$$u(X, T_c) = \sum_{i_j \in X \wedge X \subseteq T_c} u(i_j, T_c) \quad (2)$$

$$u(X) = \sum_{X \subseteq T_c \wedge T_c \in TD} u(X, T_c) \quad (3)$$

The overall utility of an itemset (Equation 3) is used as the fitness function for the HC and SA algorithms proposed in this paper.

For a transaction  $T_c$ , the *transaction utility* ( $TU$ ) is defined as  $TU(T_c) = u(T_c, T_c)$ . The *minimum utility threshold* ( $\delta$ ) that is set by the user is the percentage of the sum of all  $TU$  values for the input database, whereas the *minimum utility value* is defined as:

$$min\_util = \delta \times \sum_{T_c \in TD} TU(T_c) \quad (4)$$

An itemset  $X$  is an HUI if  $u(X) \geq min\_util$ .

For search space reduction in HUIM, an upper bound on the utility of an itemset and its supersets called the *transaction-weighted utilization* ( $TWU$ ) is often used [9]. The  $TWU$  of an itemset  $X$  is the sum of the transaction utilities of all the transactions that contain  $X$ , and is defined as:

$$TWU(X) = \sum_{X \subseteq T_c \wedge T_c \in TD} TU(T_c) \quad (5)$$

An itemset  $X$  is called a *high transaction weighted-utilization itemset* (HTWUI) if  $TWU(X) \geq min\_util$ ; otherwise,  $X$  is a low transaction weighted-utilization itemset (LTWUI). An HTWUI/LTWUI with  $k$  items is called a  $k$ -HTWUI/ $k$ -LTWUI. It can be shown that the set of all HUIs is a subset of the set of HTWUIs and no LTWUI is a HUIs [9]. Hence, if an itemset is identified as a LTWUI during the search for HUIs, all its supersets can be safely ignored.

**Problem Statement:** The problem of HUIM is defined as follows [45, 46]: Given a transaction database ( $TD$ ), its profit table ( $ptable$ ) and a user-specified minimum utility threshold, the problem of HUIM is to determine all itemsets that have utilities equal to or greater than  $min\_util$ .

In the example database, the utility of item  $c$  in transaction  $T_0$  is  $u(c, T_0) = 12 \times 3 = 36$ . Similarly, the utility of itemset  $\{a, c\}$  in transaction  $T_0$  is  $u(\{a, c\}, T_0) = u(a, T_0) + u(c, T_0) = 3 \times 2 + 12 \times 3 = 42$ , and the utility of itemset  $\{a, c\}$  in the transaction database is  $u(\{a, c\}) = u(\{a, c\}, T_0) + u(\{a, c\}, T_2) + u(\{a, c\}, T_4) = 42 + 12 + 17 = 71$ . The  $TU$  of  $T_0$  is  $TU(T_0) = u(\{a, c, e\}, T_3) = 54$ . The third column in Table 1 lists the utilities of other transactions. If  $min\_util = 85$ , then  $u(\{a, c\}) < min\_util$ ,  $\{a, c\}$  is not an HUI. On the other hand, the itemset  $\{a, c\}$  is contained in transactions  $T_0$ ,  $T_2$  and  $T_4$ . Hence, the  $TWU$  of itemset  $\{a, c\}$  is calculated as  $TWU(\{a, c\}) = TU(T_0) + TU(T_2) + TU(T_4) = 122$ . If



$min\_util = 85$ , then  $\{a, c\}$  is an HTWUI.

#### 4 PROPOSED HEURISTIC ALGORITHMS FOR HUIM

The proposed HUIM-SA and HUIM-HC algorithms are heuristic-based algorithms that rely on a bitmap database representation. They are iterative algorithms that generate a population of solutions (potential HUIs), apply a pruning strategy to eliminate some solutions, and evaluate the remaining solutions using a fitness function to select HUIs. Then, current solutions are used to generate other solutions, and this process is repeated iteratively until a maximum number of iterations is reached.

Before explaining the details of the proposed algorithms, the two strategies (1) the input database representation as a bitmap and (2) the pruning strategy of promising encoding vector are first introduced in this section. Next, the the population initialization procedure is presented. Finally, all the parts are put together and the proposed HUIM-HC and HUIM-SA algorithms are described.

##### 4.1 Bitmap Representation and Promising Encoding Vector

Bitmap are an effective representation method for mining HUIs [29]. Thus, the proposed two algorithms first convert the input database  $TD$  into a bitmap. The *bitmap of  $TD$*  is an  $n \times m$  matrix of Boolean type  $B(TD)$ , where  $m$  represents the number of distinct items and  $n$  is the transaction count. The entry in  $B(D)$  that corresponds to transaction  $T_j$  ( $1 \leq j \leq n$ ) and item  $i_k$  ( $1 \leq k \leq m$ ) is denoted as  $(j, k)$ , and is stored in the  $j$ th row and  $k$ th column in  $B(D)$ . The value of  $(j, k)$  is denoted as  $B_{(j,k)}$  and defined as:

$$B_{(j,k)} = \begin{cases} 1, & \text{iff } i_k \in T_j \\ 0, & \text{otherwise} \end{cases} \quad (6)$$

In other words, the entry  $(j, k)$  of  $B(TD)$  is 1 iff the item  $i_k$  is present in the transaction  $T_j$ , otherwise this entry is set to 0. The *bitmap cover* of item  $i_k$  in  $B(TD)$ , denoted as  $Bit(i_k)$ , is the  $k$ -th column vector. This can be naturally expanded to itemsets also. The bitmap cover of an itemset  $X$  is computed as  $Bit(X) = \text{bitwise-AND}_{i \in X} (Bit(i))$ . This shows that  $X$  is also a bit vector obtained by performing the bitwise-AND operation on the bitmap covers of all items that are present in  $X$ . For two itemsets  $X$  and  $Y$ ,  $Bit(X \cup Y)$  can be calculated as  $Bit(X) \cap Bit(Y)$  (the bitwise-AND of  $Bit(X)$  and  $Bit(Y)$ ).

For example, the bitmap of the database of Table 1 is shown in Table 3. The bitmap covers of item  $a$  and item  $c$  are the column vectors  $Bit(a) = 101110$  and  $Bit(c) = 101010$ , respectively. The bitmap cover of itemset  $\{a, c\}$  is the column vector obtained by performing the bitwise-AND of  $Bit(a)$  and  $Bit(c)$ , that is  $Bit(\{a, c\}) = 101010$ .

In the proposed algorithms, each solution of a population is a potential HUI, and is represented as an *encoding vector*. Let  $z$  represents the number of 1-HTWUIs in the input database. Then, the encoding vector of a solution is composed of  $z$  bits, where each bit represents a distinct 1-HTWUI. If the  $j$ -th position of an encoding vector is set to 1, it means that the solution contains the corresponding 1-HTWUI, and if set to 0, it means that the solution does not contain it.

A concept of *promising encoding vector* [23] is used to speed up the HUIM process. It is defined as:

**Definition 4.1.** Let  $V$  represents an encoding vector that contains 0s and/or 1s and corresponds to a solution. Let the encoding vector  $(V)$  represents an itemset  $X$ . If  $Bit(X)$  only contains 0s then

Table 3. Bitmap representation of the example database

| Item   | a | b | c | d | e | f |
|--------|---|---|---|---|---|---|
| $T'_0$ | 1 | 0 | 1 | 0 | 1 | 0 |
| $T'_1$ | 0 | 1 | 0 | 1 | 1 | 1 |
| $T'_2$ | 1 | 0 | 1 | 0 | 1 | 0 |
| $T'_3$ | 1 | 0 | 0 | 1 | 0 | 1 |
| $T'_4$ | 1 | 0 | 1 | 1 | 0 | 0 |
| $T'_5$ | 0 | 1 | 0 | 1 | 0 | 1 |

$V$  is called an unpromising encoding vector (*UPEV*), otherwise  $V$  is called a promising encoding vector (*PEV*).

It is easy to see that each itemset (solution)  $X$  that is represented by an *UPEV* cannot be an HUI since an empty encoding vector indicates that the itemset do not contain any HTWUI. For such solution, the fitness value of that solution does not need to be calculated, which can greatly reduce the runtime. This technique is called the *PEV check (PEVC)* pruning strategy. Algorithm 1 presents the pseudocode of that strategy.

---

**Algorithm 1** Checking PEV

---

**Input:**  $EV$ : An encoding vector

**Output:** A PEV of  $EV$

```

1: procedure PEVC( $EV$ )
2:   Determine the number of 1's in the  $EV$ , represented as  $VN$ ;
3:   Let the  $VN$  items in  $EV$  be denoted as  $i_1, i_2, \dots, i_{VN}$ ;
4:    $XV = Bit(i_1)$ ;
5:   for  $k = 2$  to  $VN$  do
6:      $XV' = XV \cap Bit(i_k)$ ;
7:     if  $XV'$  is a UPEV then
8:        $XV' = XV$ ;
9:     Change the bit of  $i_k$  in  $EV$  from 1 to 0;
10:    end if
11:     $XV = XV'$ ;
12:  end for
13:  return  $XV$ 
14: end procedure

```

---

Algorithm 1 takes as input an encoding vector  $EV$  and returns a *PEV*. The algorithm first determines the total number of 1s in the encoding vector ( $EV$ ) and identifies which items are represented by 1s in the  $EV$ . Then, a variable  $XV$  is created to store the bitwise-AND operation of all bitmap covers of items in  $EV$ . This variable is initialized with the bitmap cover of the first item in  $EV$ . Then, a for loop is done over the remaining items of  $EV$ . For each such item  $i_k$ , the bitwise-AND operation is applied on  $XV$  with the bitmap cover of  $i_k$ . If the resulting bit vector is a *UPEV* then the item is not kept in the final bit vector, and the result of the bitwise-AND operation is reverted. This is the application of the *PEVC* pruning strategy. Then, after the for loop has ended, the encoding vector  $XV$  is returned. In the case where  $EV$  is a *UPEV*, then Algorithm 1 returns a *PEV* that is part of  $EV$ . Otherwise,  $EV$  remains unchanged.



Every newly generated solution goes through this strategy to make sure that the solution is actually present in the database.

## 4.2 Population Initialization

The initial population for both HC and SA is first initialized randomly with *PS* solutions (where *PS* is an integer parameter). Algorithm 2 lists the population initialization procedure.

---

### Algorithm 2 Population Initialization

---

**Input:** *TD*: Transaction database,

*PS*: Size of population

**Output:** First population of solutions

```

1: procedure INIT()
2:   Scan TD one time to identify all 1-HTWUIs and remove 1-LTWUIs;
3:   Represent TD as a bitmap;
4:   for i = 1 to PS do
5:     Generate a random number numi;
6:     Generate Veci with numi bits set to 1; ▷ using Equation 7
7:     if numi > 1 then
8:       Veci = PEVC(Veci);
9:     end if
10:  end for
11: end procedure

```

---

The database is first scanned in Algorithm 2 to find all 1-HTWUIs and then remove 1-LTWUIs since they cannot be part of any HUIs. Then, the database is transformed into a bitmap. Thereafter, a for loop generates the initial individuals one by one and each individual is assigned a random number of 1s in the *i*th bit vector, where *num<sub>i</sub>* is an integer between 1 and |1-HTWUIs|. A bit vector is generated that contains *num<sub>i</sub>* 1s. The probability that the bit corresponding to *i<sub>j</sub>* will be set to 1 is determined with the following formula:

$$P_j = \frac{TWU(i_j)}{\sum_{k=1}^{|1-HTWUIs|} TWU(i_k)} \quad (7)$$

From Equation (7), it is clear that the high TWU of a 1-HTWUI gives it a higher selection probability in a solution of the first population. The pruning strategy of *PEVC* in algorithm 2 is performed only when *num<sub>i</sub>* > 1. This is due to the fact that each bit in a bit vector corresponds to a 1-HTWUI, so each 1-HTWUI is certainly contained in one or more transactions. Therefore, a bit vector of this kind is clearly a *PEV*.

For instance, consider the database of Table 1 and profit values for items of Table 2. The obtained TWU of each item after the first database scan are listed in Table 4.

Table 4. TWU of each item

| Item    | a   | b  | c   | d   | e   | f   |
|---------|-----|----|-----|-----|-----|-----|
| TWU     | 137 | 76 | 122 | 143 | 117 | 118 |
| 1-HTWUI | Yes | No | Yes | Yes | Yes | Yes |

If  $\min\_util = 85$ , then  $TWU(b) = 76 < \min\_util$ . Thus, item  $b$  is removed/deleted. Table 5 lists the reorganized transactions in the database and their respective TUs. Note that the item  $b$  has been removed from transactions  $T_1$  and  $T_5$ . Moreover, its utility have been removed from the TUs of two transactions ( $T_1$  and  $T_5$ ). The result for the created bitmap representation of the reorganized database is shown in Table 6.

Table 5. Reorganized transaction database

| TID    | Transactions          | TU |
|--------|-----------------------|----|
| $T'_0$ | (a, 3) (c, 12) (e, 3) | 54 |
| $T'_1$ | (d, 2) (e, 1) (f, 5)  | 19 |
| $T'_2$ | (a, 3) (c, 2) (e, 1)  | 16 |
| $T'_3$ | (a, 2) (d, 2) (f, 1)  | 15 |
| $T'_4$ | (a, 1) (c, 5) (d, 7)  | 52 |
| $T'_5$ | (d, 4) (f, 2)         | 22 |

Table 6. Bitmap representation of the reorganized database

| Item   | a | c | d | e | f |
|--------|---|---|---|---|---|
| $T'_0$ | 1 | 1 | 0 | 1 | 0 |
| $T'_1$ | 0 | 0 | 1 | 1 | 1 |
| $T'_2$ | 1 | 1 | 0 | 1 | 0 |
| $T'_3$ | 1 | 0 | 1 | 0 | 1 |
| $T'_4$ | 1 | 1 | 1 | 0 | 0 |
| $T'_5$ | 0 | 0 | 1 | 0 | 1 |

### 4.3 HC and SA for HUIM

This section presents the proposed HUIM-HC algorithm and then the HUIM-SA algorithm.

### 4.4 HUIM-HC

Hill climbing (HC) is a heuristic and search based method used to solve optimization problems. The main steps in HC include: (1) generate population, (2) select candidate solutions (called chromosomes) from the population and (3) population exploration. In the population exploration phase, HC tries to find the solutions in the population that are better than previously selected solutions. For the problem of HUIM, the HC algorithm finds sufficiently good solutions (HUIs) for a given database and a heuristic function  $f$ . Here the heuristic function  $f$  is the utility of itemsets whose value is greater than  $\min\_util$ . Algorithm 3 presents the pseudocode of the proposed HC algorithm for HUIM.

The algorithm takes as input a database, the minimum utility threshold and a maximum number of generations. The algorithms first creates an initial population by calling the *INIT()* procedure. Then, a variable *gene* is set to 1 to remember that this is the first population, and a set *SHUI* is initialized as the empty set, which is used for storing HUIs (SHUI) that will be found. A while loop is then repeated to discover HUIs, population by population, until a maximum number of generations is reached. A for loop iterates over each chromosome (solution)  $C$  of the population. The function *IS()* transform the solution  $C$  into an itemset  $X$  by adding each item in  $C_i \in C$  to  $X$  if its value is 1. Then, if  $X$  is a HUI and it has not already been discovered, it is stored in the set *SHUI*.

**Algorithm 3** HUIM-HC**Input:** *TD* (Database), *min\_util* (Minimum utility), *max\_gen* (Maximum generations)**Output:** *HUIs* (High utility itemsets)

---

```

1: INIT();
2: gene ← 1;
3: SHUI ← ∅;
4: while gene < max_gene do
5:   for each Ci do
6:     X ← IS(Ci);
7:     if min_util ≤ u(X) ∧ X ∉ SHUI then
8:       SHUI ← SHUI ∪ X;
9:     end if
10:  end for
11:  GN();
12:  Select some Ci's from SHUI, represented as bit vectors;           ▶ using Equation 8
13:  Replace randomly selected HUIs in the current population with Ci's;
14:  gene ← gene + 1
15: end while
16: Output all HUIs;

```

---

Thereafter, the neighbor procedure *GN*() is called to generate the next population. Then Equation 8 is used to select some already discovered *HUIs*. Selected *HUIs* are represented as bit vectors and replaces some randomly selected solutions in the new population. This improves the population diversity. When the maximum number of generations is reached, all discovered *HUIs* are output.

$$P_i = \frac{fitness_i}{\sum_{j=1}^{|SHUI|} fitness_j} \quad (8)$$

$|SHUI|$  in Equation 8 represents the total number of already discovered *HUIs*, and *fitness<sub>i</sub>* represents the fitness value of the *i*th *HUI* in the population.

The Neighbor procedure that is used for the generation of the next population is listed in Algorithm 4. In this procedure, *PS<sub>new</sub>*, that represents the total number of chromosomes in the new population, is initialized to 0. On the basis of current *PS* chromosomes, the main loop in this procedure generates the next population. First, a chromosome is selected from the current population and the value of a random location (*j*) is changed. In this way, this procedure is able to find a neighbor of the selected chromosome. It is important to mention here that the get neighbor procedure of HC and the standard mutation (SM) operator of GA are quite similar [47]. In SM, a location is first randomly selected and its value is changed from its original value with a probability, called mutation probability (*p<sub>m</sub>*). Here we do not use any probability and every selected chromosome is processed to get its neighbor.

**4.4.1 Illustrated Example for HUIM-HC.** The database and profit table in Table 1 and Table 2 are used respectively to explain the working of HUIM-HC. Let *min\_util* = 56 and all the 1-LTWUs are removed. Let the population size (*PS*) is 3. After transforming the database into the form of Table 6, we know that the size of each chromosome is 5 (it is equal to the number of discovered 1-HTWUs). Thus, in the bit vector, there are five bits for chromosome encoding. At the start, the *SHUI* set is

**Algorithm 4** Get Neighbor**Input:** Current population**Output:** Next population

```

1: procedure GN()
2:    $PS_{new} = 0$ ;
3:   while  $PS_{new} < PS$  do
4:     Select one chromosome  $C_i$  from SN; ▷ using Equation 8
5:      $j \leftarrow \text{randomint}(0, \text{size})$ ;
6:     if  $C_i(j) == 0$  then
7:        $C_i(j) = 1$ ;
8:     else
9:        $C_i(j) = 0$ ;
10:    end if
11:     $C_k = \text{PEVC}(C_i)$ ;
12:     $PS_{new} \leftarrow PS_{new} + 1$ ;
13:  end while
14: end procedure

```

initialized to an empty set. A random number is generated to create the first chromosome. Assume that the generated number is 4. This number basically shows the number of 1s in the chromosome. To calculate which bits are set to 1, Equation 7 is used. Let the generated bit vector for the first chromosome,  $C_1$ , is 11110. The other two chromosomes,  $C_2$  and  $C_3$  can be obtained using the same method. The bit vectors of the three chromosomes in the first population are shown in Figure 1(a).

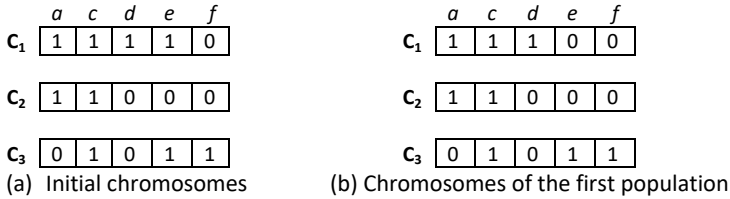


Fig. 1. Chromosomes population-wise

The first chromosome  $C_1$  represents itemset  $\{a, c, d, e\}$ . The XV is initialized by  $Bit(a)$ , according to Algorithm 1, so  $XV = 101110$  and  $XV \cap Bit(c) = 101110 \cap 101010 = 101010$ . As this obtained bit vector is a PEV, so XV is updated to 101010. Next,  $XV \cap Bit(d) = 101010 \cap 010111 = 000010$ . Again the obtained bit vector is a PEV, so XV is updated to 000010. Next,  $XV \cap Bit(e) = 000010 \cap 111000 = 000000$ . As XV is a UPEV, the item  $e$  is deleted from  $C_1$ , and XV retains the value 000010. Thus,  $C_1$  is 11100 that represents the itemset  $\{a, c, d\}$ , and it is present in  $T'_4$ . However,  $\{a, c, d\}$  is not an HUI, as  $u(acd) = 52 < \min\_util$ .

Similarly, the chromosome  $C_2$  is also a PEV that represents the itemset  $\{a, c\}$ . Itemset  $\{a, c\}$  is present in three transactions ( $T'_0$ ,  $T'_2$  and  $T'_4$ ), and  $u(ac) = 71 > \min\_util$ . Thus,  $SHUI = \{ac : 71\}$ . The number after the colon is for the utility value of itemset. The chromosome  $C_3$  that represents the itemset  $\{c, e, f\}$  is not an HUI. Therefore, the SHUI remains the same. Till now, three chromosomes are present in the first population (shown in Figure 1(b)).

Suppose that  $C_3$  is selected at first and the fifth bit in the bit vector is selected randomly. Through the *Neighbor* procedure, fifth bit is changed from 1 to 0. Thus,  $C_3$  becomes 01010 that represents

the itemset  $\{c, e\}$ . This is a PEV and is present in two transactions ( $T'_0$  and  $T'_2$ ). As  $u(ce) = 58 > \min\_util$ , so  $SHUI$  is updated and new  $SHUI = \{ac : 71, ce : 58\}$ . After the generation of new chromosomes,  $PS_{new} = 2$ . As  $PS_{new} < PS$ , so some more chromosomes, from the current population, will be selected and the above process is repeated until  $PS_{new} \geq PS$ .

Assume that no new HUIs are generated by the other chromosomes and  $SHUI$  is still  $\{ac : 71, ce : 58\}$ . According to Algorithm 3, some HUIs will be selected using Equation 8. That is,  $\{a, c\}$  has the highest probability of selection while  $\{c, e\}$  has the lowest. Selected HUIs are used to replace some randomly selected chromosomes from the second population. This whole process continues for the new population until the termination condition is reached.

#### 4.5 HUIM-SA

Simulated annealing (SA) is a probabilistic-based metaheuristic method for solving the black box global optimization problems. SA is based on the notion of physical annealing: the process of heating and then slowly cooling a metal to get a strong crystalline. SA consists of four main steps, that are: (1) problem configuration, (2) neighborhood configuration, (3) objective function, and (4) cooling/annealing process. As common in metaheuristic algorithms, SA starts by first generating a random initial solution. SA makes progress in each iteration by replacing the current solution by a random "neighbor" solution. The neighbor solution is selected using a probability that depends on the difference between the corresponding function values and a global parameter  $T$  (called the temperature). In each iteration, the value of  $T$  is gradually decreased. SA and HC algorithms are very similar with one main difference: at high temperature, SA switches to a worse neighbor. Algorithm 5 lists the proposed SA algorithm for HUIM.

Just like the HC algorithm, an initial population is first created.  $SHUI$ , the set that stores the HUIs, is initialized to the empty set. The while loop discovers the set of HUIs population by population. The difference is that the while loop contains the *temp*, *min\_temp* and *alpha* ( $\alpha$ ) parameters. Moreover, with *acceptance* probability, there is the chance that HUIs will be added to  $SHUI$ . SA checks whether the new solution (newly find HUI) is better than the current solution. SA may select the new solution in case the new solution is not better than the current solution. This is achieved with *acceptance probability* that governs whether to switch to the worst solution or not. The reason for this is to avoid staying into a local optimum and explore other solutions. Although the idea of selecting worse solution seems awkward or sometimes unacceptable, it can lead SA to reach the global optimum. The *acceptance probability* is computed by using the acceptance rate (AR) formula:

$$AR = \exp\left(\frac{T}{1+T}\right) \quad (9)$$

The importance of the *acceptance probability* in HUIM-SA is examined in the next section, where the performance of HUIM-SA that implements the *acceptance probability* parameter is compared with HUIM-SA without this parameter on different datasets for mining HUIs.

**4.5.1 Illustrated Example for HUIM-SA.** The execution of HUIM-SA is illustrated using the same example as for HUIM-HC. The first population is  $C_1 = 11100$ ,  $C_2 = 11000$  and  $C_3 = 01011$ . Compared to HUIM-HC, this example differs in two aspects. First, generations and maximum generations in HC are replaced by temperature and minimum temperature in SA. Second, a probability (called *acceptance probability*) is used in SA that can select those HUIs whose values are less than *min\_util*.

To see how the next population is generated, let us consider  $C_1$  and suppose that the first bit is selected randomly in  $C_1$  and is changed from 1 to 0. With this change,  $C_1 = 01100$  and it is a PEV

---

**Algorithm 5** HUIM-SA
 

---

**Input:**  $TD$  (Database),  $min\_util$  (Minimum utility),  $Temp$ ,  $Min\_Temp$ ,  $\alpha$ 
**Output:**  $HUIs$  (High utility itemsets)

```

1: INIT();
2:  $T \leftarrow Temp$ ;
3:  $MT \leftarrow Min\_Temp$ ;
4:  $SHUI \leftarrow \emptyset$ ;
5: while  $T > MT$  do
6:   for each  $C_i$  do
7:      $X = IS(C_i)$ ;
8:     if  $u(X) \geq min\_util \wedge X \notin SHUI$  then
9:        $SHUI \leftarrow SHUI \cup X$ ;
10:    end if
11:     $ar = \exp(\frac{T}{1+T})$ ;
12:    if  $ar > \text{randomuniform}(0,10)$  then
13:       $SHUI \leftarrow SHUI \cup X$ ;
14:    end if
15:  end for
16:  GN();
17:  Select some  $C_i$ 's from  $SHUI$ , represented as bit vectors; ▷ using Equation 8
18:  Replace randomly selected  $HUIs$  in the current population with  $C_i$ 's;
19:   $T \leftarrow T \times \alpha$ ;
20: end while
21: Output all  $HUIs$ ;
```

---

and  $u(cd) = 50 < min\_util$ . Thus,  $\{c, d\}$  is not an HUI. The same procedure can be used to obtain new  $C_2$  and  $C_3$ . After the second iteration, suppose that  $SHUI = \{ac : 71, ce : 58\}$ . In SA,  $C_1$  that represents HUI  $\{c, d\}$  will be in the  $HUIs$  if the probability calculated using Equation 9 is greater than a random number generated in the range (0, 10). The above process continues till the value of  $T$  becomes less than or equal to  $MT$ .

## 5 RESULTS AND DISCUSSION

This section presents the experimental evaluation of the proposed algorithms and a discussion of results. In the experiments, the proposed HUIM-SA and HUIM-HC were compared with six state-of-the-art algorithms for HUIM using heuristic and evolutionary-based algorithms, that are: HUIF-GA [23], HUIF-PSO [23], HUIF-BA [23], HUPE<sub>UMU</sub>-GARM (HUIM-GA) [18], HUIM-BPSO<sub>Sig</sub> (HUIM-BPSOS) [20] and HUIM-BPSO [19]. The main reason to use these six algorithms for comparison is that their codes are publicly available in the SPMF tool [48].

The experiments were carried out on a computer with an 8-Core 3.600 GHz CPU and 64 GB RAM running Windows 10 (64-bit). The programs for HUIM-HC and HUIM-SA were developed in Java. Six datasets were used to evaluate the performance of HUIM-HC and HUIM-SA. Those datasets are standard benchmark datasets. The sparse dataset Foodmart and one dense dataset Ecommerce have real utility values, while the remaining dense four datasets have synthetic utility values. The main characteristics of the six datasets are presented in Table 7.



Table 7. Characteristics of the datasets

| Dataset       | Avg. Trans. Len. | #Items | #Trans | Type   |
|---------------|------------------|--------|--------|--------|
| Foodmart      | 4.42             | 1,559  | 4,141  | Sparse |
| Chess         | 37               | 75     | 3,196  | Dense  |
| Mushroom      | 23               | 119    | 8,416  | Dense  |
| Accidents_10% | 34               | 468    | 34,018 | Dense  |
| Connect       | 43               | 129    | 67,557 | Dense  |
| Ecommerce     | 11.71            | 3,468  | 14,975 | Dense  |

All the datasets were downloaded from the SPMF data mining library [48]. The Foodmart dataset contains customer transactions from a retail store. The Chess dataset originates from the chess game steps. The Mushroom dataset contains different mushrooms species and their characteristics, such as habitat, shape and odor. The Accident dataset contains anonymized traffic accident data. Similar to previous studies [19, 23, 24], only 10% of this dataset was used in experiments. The Connect dataset is also derived from game steps. The Ecommerce dataset contains customer transactions from December 1 2020 to December 09 2021 of a UK-based online store.

For all experiments, the termination criterion for all algorithms, except HUIM-SA, was set to 10,000 iterations and the initial population size was set to 30. HUIM-SA terminates when  $T < MT$ . Moreover, for HUIM-SA, the values for  $temp$  ( $T$ ),  $min\_temp$  ( $MT$ ) and  $\alpha$  were set to 100,000, 0.00001, and 0.9993 respectively. Moreover, the calculated AR value (using Equation 9) was compared with a random number generated within the range (2.8, 3.2). This range was selected after doing some preliminary tests with the values of  $temp$ ,  $min\_temp$  and  $\alpha$ .

## 5.1 Runtime

Experiments were first carried out to evaluate the efficiency of the algorithms in terms of runtime. The runtime was measured while varying the minimum utility value for each dataset. Figure 2 shows the execution time for the six datasets.

It is observed that the designed HUIM-SA and HUIM-HC algorithms were faster than other evolutionary-based HUIM algorithms except for the Foodmart database, where HUIF-PSO was faster than HUIM-SA. On Ecommerce dataset, the runtime of HUIF-PSO was almost similar to HUIM-SA. Overall, both algorithms demonstrated relatively steady execution times on the six datasets. The main reason behind this is the use of strategies such as bitmap representation and promising encoding vector. Moreover, some procedures of HUIF-based frameworks such as *BittDiff*, that stores the different bits in two chromosomes, were not used in HUIM-HC and HUIM-SA.

HUIM-SA was always slower than HUIM-HC due to the extra annealing process of HUIM-SA. For the Foodmart dataset, the average execution time for HUIM-BPSO and HUIM-BPSOS were high, approximately 527 and 7,270 seconds respectively. That is why they are not added in the graph for Foodmart. HUIM-GA takes more than three hours on the Foodmart and Ecommerce dataset. Moreover, the execution times for these three algorithms (HUIM-GA, HUIM-BPSO and HUIM-BPSOS) were more than 3 hours for the Accidents and Connect datasets. For the Ecommerce dataset, the average execution time for HUIM-BPSO was high, approximately 6200 seconds. Moreover, the execution time for HUIM-BPSOS was more than 3 hours on the Ecommerce datasets.

In literature, traditional algorithms for HUIM are rarely compared with evolutionary/heuristic-based algorithms for runtime. For example, [23] compared the IHUP [10] and UP-Growth [11] with HUIF-GA, HUIF-PSO and HUIF-BA, respectively, on four datasets. Similarly, [41] compared one improved GA (HUIM-IGA) with traditional exact algorithms for HUIM. Obtained results for runtime

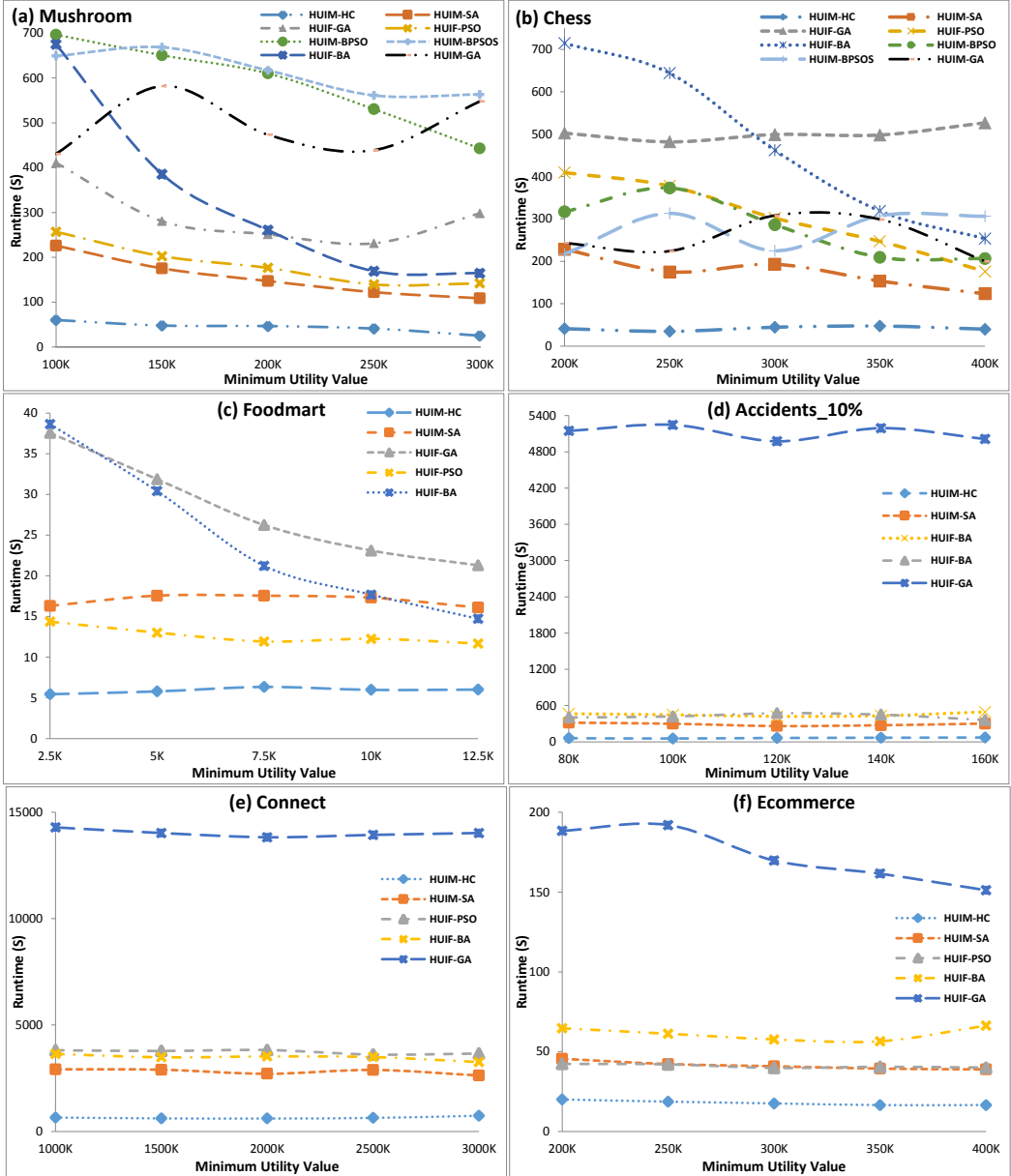


Fig. 2. Execution times of the compared algorithms on six datasets

clearly indicate that traditional exact algorithms consumed more time than evolutionary/heuristic algorithms for HUIM. Moreover, the works [18–20, 24, 44] did not compared the exact algorithms with evolutionary algorithms for execution time.

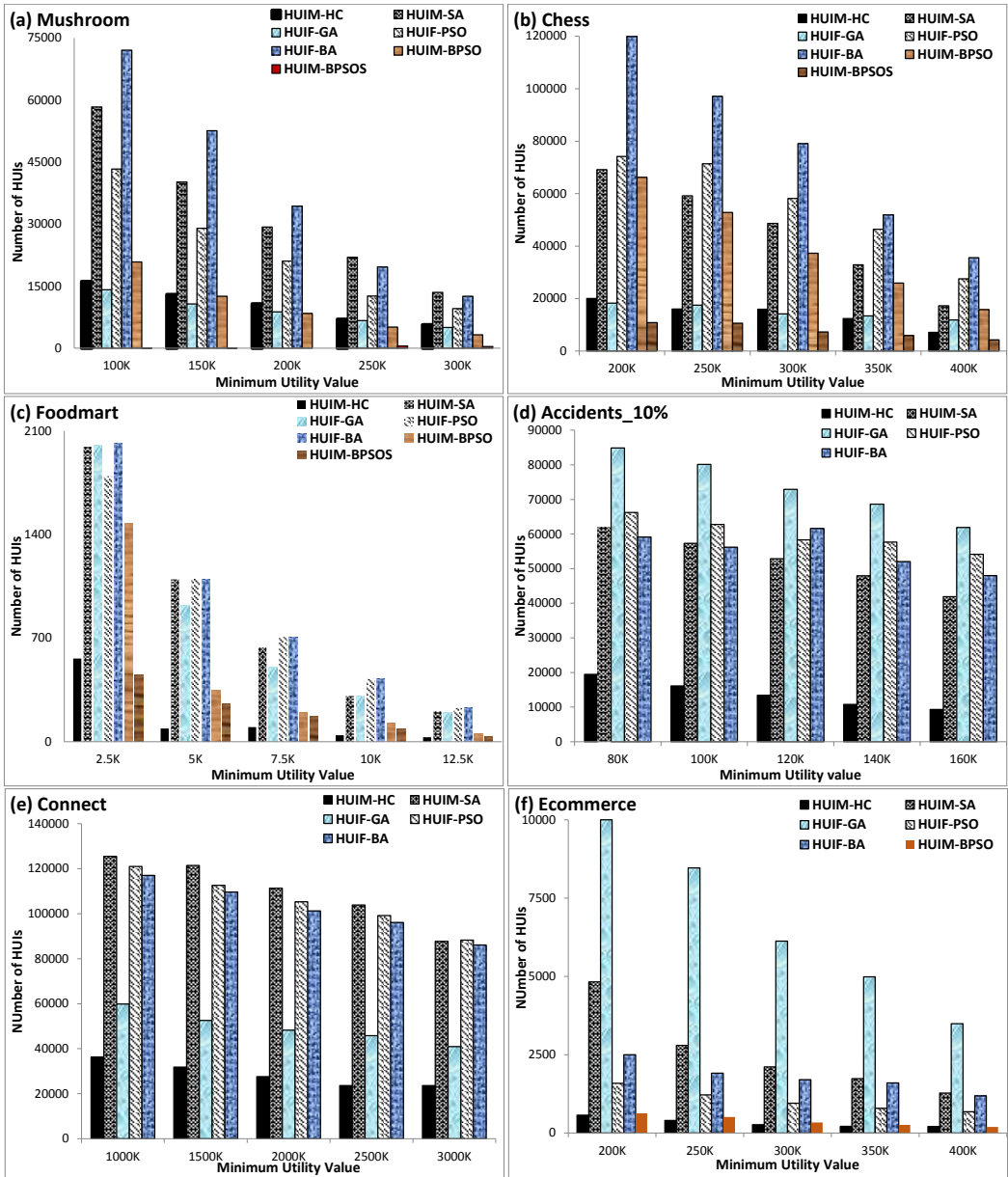


Fig. 3. Number of discovered HUIs

## 5.2 Discovered HUIs

In this section, the number of HUIs discovered by the compared algorithms for the same six datasets and parameter values is compared. Evolutionary/heuristic-based algorithms cannot ensure the discovery of all itemsets within a certain number of iterations. The works [19, 20, 24] compared the performance of HUIM-BPSO, HUIM-BPSOS and HUIM-ABC, respectively, with the TWU model [9] (the Two-Phase algorithm for HUIM) for discovered HUIs. Similarly, [41] compared

the number of HUIs mined by various evolutionary/heuristic-based algorithms with traditional exact approaches. Obtained results indicate that traditional exact algorithms find more HUIs than evolutionary/heuristic-based algorithms. Thus, we compared the number of discovered HUIs by evolutionary/heuristic-based algorithms for HUIM. The comparison results are shown in Figure 3.

Table 8. Percentage (%) of mined HUIs by five algorithms

| Dataset   | MUV   | HUIM-HC | HUIM-SA | HUIF-GA | HUIF-PSO | HUIF-BA |
|-----------|-------|---------|---------|---------|----------|---------|
| Mushroom  | 100K  | 22.3    | 80.9    | 19.6    | 60       | 100     |
|           | 150K  | 24.6    | 76.3    | 20.3    | 55.1     | 100     |
|           | 200K  | 31.2    | 85.2    | 25.6    | 61.3     | 100     |
|           | 250K  | 35.8    | 100     | 33.9    | 64.4     | 100     |
|           | 300K  | 45.1    | 100     | 40      | 76.4     | 100     |
| Chess     | 200K  | 16.7    | 57.7    | 15.2    | 61.9     | 100     |
|           | 250K  | 16.5    | 60.9    | 18      | 73.5     | 100     |
|           | 300K  | 20.2    | 61.5    | 17.8    | 73.5     | 100     |
|           | 350K  | 23.8    | 63.4    | 25.8    | 89.4     | 100     |
|           | 400K  | 20      | 48.4    | 33.2    | 77.4     | 100     |
| Foodmart  | 2.5K  | 27.8    | 98.8    | 99.3    | 88.9     | 100     |
|           | 5K    | 8.2     | 100     | 84      | 100      | 100     |
|           | 7.5K  | 13.9    | 89.8    | 71.7    | 100      | 100     |
|           | 10K   | 10.6    | 72.8    | 73.5    | 100      | 100     |
|           | 12.5K | 13.1    | 90.3    | 85.9    | 100      | 100     |
| Accidents | 80K   | 22.9    | 72.8    | 100     | 78       | 69.6    |
|           | 100K  | 20      | 71.5    | 100     | 78.3     | 70.1    |
|           | 120K  | 18.4    | 72.4    | 100     | 79.9     | 84.4    |
|           | 140K  | 15.6    | 69.7    | 100     | 84       | 75.7    |
|           | 160K  | 15      | 67.7    | 100     | 87.4     | 77.5    |
| Connect   | 1000K | 28.9    | 100     | 47.6    | 96.3     | 93.2    |
|           | 1500K | 26.2    | 100     | 43.2    | 92.6     | 90.2    |
|           | 2000K | 24.7    | 100     | 43.3    | 94.5     | 91      |
|           | 2500K | 22.7    | 100     | 44.1    | 95.4     | 92.5    |
|           | 3000K | 26.9    | 100     | 46.7    | 100      | 98.2    |
| Ecommerce | 200K  | 5.6     | 48.2    | 100     | 15.8     | 24.9    |
|           | 250K  | 4.6     | 33      | 100     | 14.3     | 22.5    |
|           | 300K  | 4.3     | 34.4    | 100     | 15.5     | 22.7    |
|           | 350K  | 4.2     | 34.7    | 100     | 15.8     | 31.9    |
|           | 400K  | 5.8     | 36.5    | 100     | 19.4     | 33.8    |

Some interesting observations are made about the HUIs discovered by each algorithm on the six datasets. On the Mushroom dataset, the performance of HUIM-SA was better than other algorithms, except HUIF-BA. On the Chess dataset, only HUIF-BA and HUIF-PSO performed better than HUIM-SA. Whereas, the performance of HUIM-SA was almost similar to other HUIF-based algorithms for Foodmart and its performance was relatively low on the Accidents dataset. On Connect, the performance of HUIM-SA was relatively better than other algorithms. On Ecommerce dataset, the performance of HUIM-SA was better than other algorithms, except HUIF-GA. HUIM-HC performed poorly on all dataset if compared to HUIF-GA, HUIF-PSO and HUIF-BA, but its performance was better than HUIM-BPSO and HUIM-BPSOS. The main reason for this is that HUIM-HC tends to get

stuck at local optima that may cause premature convergence. HUIM-GA results were not included in Figure 3 as those were the worst results among all algorithms. It was able to discover 85 HUIs in the Mushroom dataset with  $min\_util = 100K$  and 159 HUIs for the Chess dataset with  $min\_util = 200K$ . We found that the performance of HUIM-HC and HUIM-SA decreases with an increase of  $min\_util$  and for very high  $min\_util$  value, they tend to perform poorly.

Results in Figure 3 indicate that the three algorithms based on the HUIF framework performed better than HUIM-GA, HUIM-BPSO and HUIM-BPSOS. Table 8 summarizes the result of Figure 3 by comparing the percentage of discovered HUIs by five algorithms on six datasets. Through percentage comparison, we can see the difference in the numbers of HUIs mined by different evolutionary/heuristic-based approaches.

The notion of *acceptance probability* along with the annealing process distinguish SA from other evolutionary and search methods. The *acceptance probability* allows SA to accept a new solution (possible HUI in this work) obtained with the *Neighbor* procedure that is actually worst than the present solution (HUI). The main reason for this is that there is always a possibility that the worst solution could lead the SA to the global optimum. Next we checked whether removing the acceptance probability from SA has any effect on the number of discovered HUIs. HUIM-SA without the acceptance probability is named HUIM-SA+. Table 9 compares HUIM-SA and HUIM-SA+ on different datasets in terms of number of discovered HUIs.

Table 9. Discovered HUIs by HUIM-SA and HUIM-SA+

| Dataset  | MUV   | HUIM-SA | HUIM-SA+ | Dataset   | MUV  | HUIM-SA | HUIM-SA+ |
|----------|-------|---------|----------|-----------|------|---------|----------|
| Mushroom | 100K  | 58,354  | 55,486   | Chess     | 200K | 69,172  | 53,457   |
|          | 150K  | 40,175  | 38,183   |           | 250K | 59,168  | 50,448   |
|          | 200K  | 29,277  | 27,082   |           | 300K | 48,642  | 44,146   |
|          | 250K  | 21,967  | 19,379   |           | 350K | 32,905  | 29,885   |
|          | 300K  | 13,482  | 11,341   |           | 400K | 17,251  | 13,985   |
| Foodmart | 2.5K  | 1,992   | 1,426    | Accidents | 80K  | 61,846  | 43,013   |
|          | 5K    | 1,096   | 728      |           | 100K | 57,295  | 37,913   |
|          | 7.5K  | 636     | 271      |           | 120K | 52,856  | 29,472   |
|          | 10K   | 309     | 156      |           | 140K | 47,912  | 26,923   |
|          | 12.5K | 206     | 99       |           | 160K | 41,925  | 23,634   |
| Connect  | 1000K | 125,512 | 116,579  | Ecommerce | 200K | 4,826   | 2,459    |
|          | 1500K | 121,458 | 101,128  |           | 250K | 2,795   | 1,318    |
|          | 2000K | 111,287 | 91,978   |           | 300K | 2,109   | 1,013    |
|          | 2500K | 103,825 | 82,081   |           | 350K | 1,734   | 941      |
|          | 3000K | 87,692  | 75,226   |           | 400K | 1,276   | 627      |

From Table 9, we can see that HUIM-SA+ finds less HUIs than HUIM-SA for all datasets. The difference between the number of discovered HUIs for the Mushroom dataset is low as compared to on the other five datasets. Thus, *acceptance probability* indeed allows SA to find more HUIs by making the SA to avoid local optimum by exploring other (not so-well) solutions. Note that the time spent by HUIM-SA and HUIM-SA+ to find HUIs in all datasets was almost the same with negligible difference.

### 5.3 Convergence

This section presents an evaluation of the convergence for all the datasets. Obtained results are shown in Figure 4.

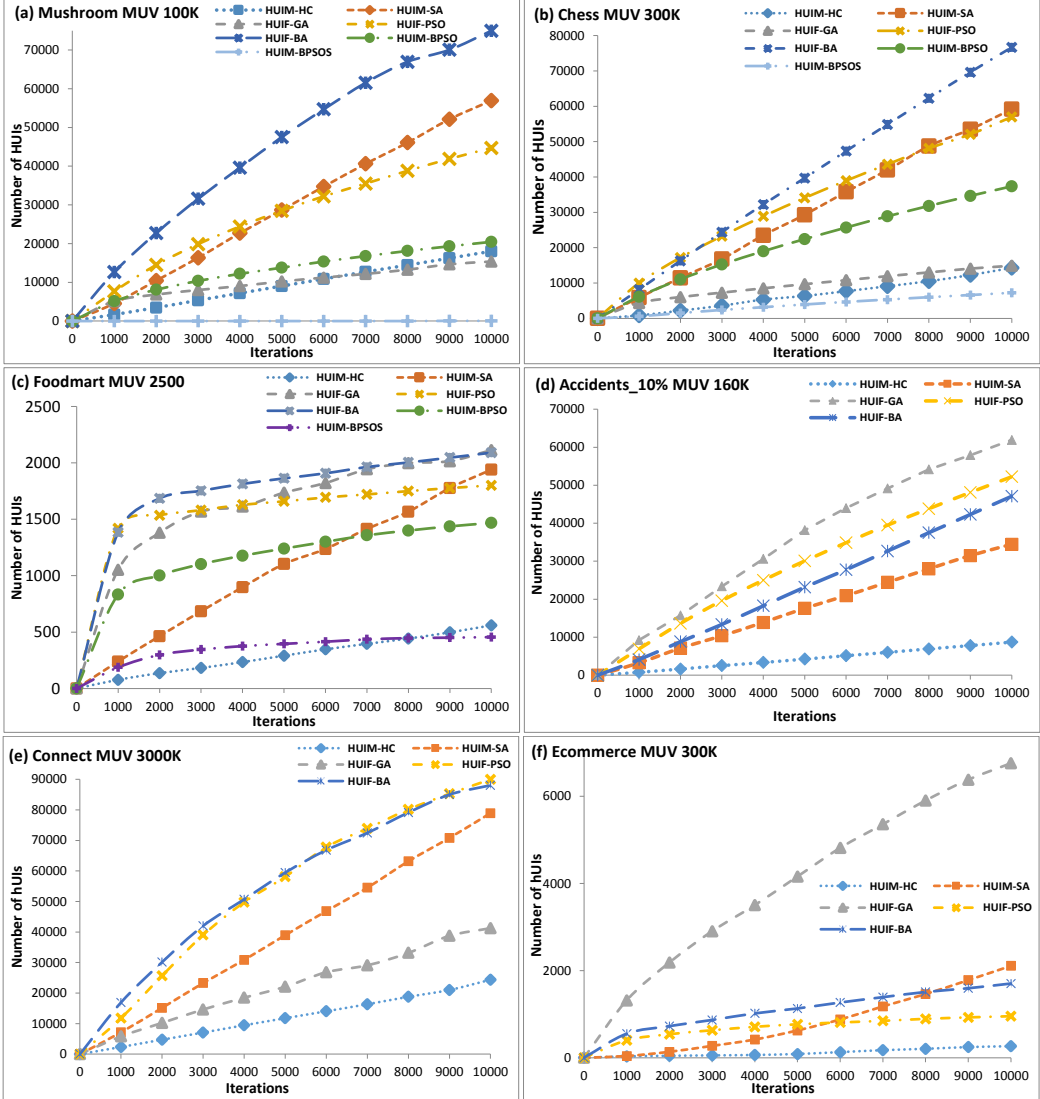


Fig. 4. Convergence performance of algorithms

The convergence speed of HUIM-SA was linear for all datasets. Whereas the convergence speed for HUIF-BA was faster at the start, it decreases with the number of iterations. HUIM-SA performance was better on datasets that contain a large number of transactions and a small number of items (such as Connect) and on datasets with fewer transaction but with a high number of items such as (Foodmart). For datasets with less transactions and fewer items (Mushroom and Chess), the performance is comparable to other HUIF algorithms. However, the performance of HUIM-SA is poor on datasets that contains a large number of transactions with high number of items (such as Accidents). On Ecommerce dataset, it can be seen that HUIM-SA performance was low at the start. However, with increase in iterations, its performance gets better.



## 6 CONCLUSION

Two (meta)heuristic based algorithms were proposed in this paper to mine HUIs in large datasets. The first algorithm called HUIM-HC was based on Hill Climbing and the second algorithm called HUIM-SA was based on Simulated Annealing. Both algorithms used bitmap representation and promising encoding vector for search space pruning. Moreover, during evolution, discovered HUIs in the current population were used as target values for the next population. Experimental results showed that HUIM-HC and HUIM-SA were better than existing algorithms execution time wise and HUIM-SA performed similar to other algorithms in terms of discovered HUIs. Moreover, the convergence analysis showed that HUIM-SA was evolving linearly during the evolution process. There are several directions for future work, some of which include:

- Performing more experiments with headless chicken macromutation [49] to investigate the usefulness of crossover operators in GAs for HUIM. Some preliminary work in this regard can be found in [50].
- Implementing the PSO algorithms with headless chicken macromutation [51] for HUIM and compare the results with standard PSO algorithms.
- Implementing HUIM-HC and HUIM-SA for high average-utility itemset mining (HAUIM) problem [52].
- Parallel implementation of existing evolutionary-based HUIM algorithms.

## REFERENCES

- [1] P. Fournier-Viger, J. C. W. Lin, R. U. Kiran, Y. S. Koh and R. Thomas. 2017. A survey of sequential pattern mining. *Data Sci. Patt. Recogn.* 1, 1 (2017), 54-77.
- [2] J. M. Luna, P. Fournier-Viger and S. Ventura. 2019. Frequent itemset mining: A 25 years review. *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.* 9, 6, (2019), e1329.
- [3] C. Zhang and S. Zhang. 2002. *Association Rule Mining, Models and Algorithms*, Springer.
- [4] P. Fournier-Viger, J. C. W. Lin, T. Truong-Chi and R. Nkambou. 2019. A survey of high utility itemset mining. In *High-Utility Pattern Mining: Theory, Algorithms and Applications*, 1-45. Springer.
- [5] L. Ni, W. Luo, N. Lu, W. Zhu. 2020. Mining the local dependency itemset in a products network. *ACM Trans. Manag. Inf. Syst.* 11, 1, 3 (2020).
- [6] Mo. Zihayat, H. Davoudi and A. An. 2016. Top-k utility-based gene regulation sequential pattern discovery. In *Proceedings of International Conference on Bioinformatics and Biomedicine*. 266-273.
- [7] B. E. Shie, J. H. Cheng, K. T. Chuang and V. S. Tseng. 2012. A one-phase method for mining high utility mobile sequential patterns in mobile commerce environments. In *Proceedings of International Conference on Industrial Engineering and Other Applications of Applied Intelligent Systems*. 616-626.
- [8] W. Gan, J. C. W. Lin, H. C. Chao, P. Fournier-Viger, X. Wang, P. S. Yu. Utility-driven mining of trend information for intelligent system. 2020. *ACM Trans Manag. Inf. Syst.* 11, 3, 14 (2020).
- [9] Y. Liu, W. Liao and A. N. Choudhary. 2005. A two-phase algorithm for fast discovery of high utility itemsets. In *Proceedings of Pacific-Asia Conference on Knowledge Discovery and Data Mining*. 689-695.
- [10] C. F. Ahmed, S. K. Tanbeer, B. Jeong and Y. Lee. 2009. Efficient tree structures for high utility pattern mining in incremental databases. *IEEE Trans. Knowl. Data. Eng.* 21, 12 (2009), 1708-1721.
- [11] V. S. Tseng, C. Wu, B. Shie, and P. S. Yu. 2010. UP-Growth: An efficient algorithm for high utility itemset mining. In *Proceedings of International Conference on Knowledge Discovery and Data Mining*. 253-262.
- [12] V. S. Tseng, C. Wu, P. Fournier-Viger and P. S. Yu. 2016. Efficient algorithms for mining top-k high utility itemsets. *IEEE Tran. Knowl. Data. Eng.* 28, 1 (2016), 54-67.
- [13] P. Fournier-Viger, C. Wu, S. Zida and V. S. Tseng. 2014. FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning. In *Proceedings of International Symposium on Foundations of Intelligent Systems*. 83-92.
- [14] S. Zida, P. Fournier-Viger, J. C. W. Lin, C. Wu, V. S. Tseng. 2015. EFIM: A highly efficient algorithm for high-utility itemset mining. In *Proceedings of Mexican International Conference on Artificial Intelligence*. 530-546
- [15] S. Ventura and J. M. Luna. 2016. *Pattern Mining with Evolutionary Algorithms*. Springer.
- [16] J. M. Luna, M. Pechenizkiy, M. J. del Jesus and S. Ventura. 2017. Mining context-aware association rules using grammar-based genetic programming. *IEEE Trans. Cyber.* 48 , 11 (2017), 3030-3044.



- [17] X. Yu and M. Gen 2010. *Introduction to Evolutionary Algorithms*, Springer.
- [18] S. Kannimuthu and K. Premalatha. 2014. Discovery of high utility itemsets using genetic algorithm with ranked mutation. *Appl Artif Intell*, 28, 4 (2014), 337-359.
- [19] J. C. W. Lin, L. Yang, P. Fournier-Viger, T. Hong and M. Voznak. 2017. A binary PSO approach to mine high-utility itemsets. *Soft Comput.* 21, 17, (2017), 5103-5121.
- [20] J. C. W. Lin, L. Yang, P. Fournier-Viger, J. M. Wu, T. Hong, S. L. Wang and J. Zhan. 2016. Mining high-utility itemsets based on particle swarm optimization. *Eng Appl Artif Intell*. 55 (2016), 320-330.
- [21] K. E. Heraguemi, N. Kamel and H. Drias. 2014. Association rule mining based on bat algorithm. In *Proceedings of International Conference on Bio-Inspired Computing-Theories and Applications*. 182-186
- [22] K. E. Heraguemi, N. Kamel and H. Drias. 2016. Multi-swarm bat algorithm for association rule mining using multiple cooperative strategies. *Appl. Intell.* 45, 4 (2016), 1021-1033.
- [23] W. Song and C. Huang. 2018. Mining high utility itemsets using bio-inspired algorithms: A diverse optimal value framework. *IEEE Access*. 6 (2018), 19568-19582.
- [24] W. Song and C. Huang. 2018. Discovering high utility itemsets based on the artificial bee colony algorithm. In *Proceedings of Pacific-Asia Conference on Knowledge Discovery and Data Mining*. 3-14.
- [25] S. J. Russell and P. Norvig. 2010. *Artificial Intelligence - A Modern Approach*, Third International Edition. Pearson Education.
- [26] S. Kirkpatrick, C. D. Gelatt and M. P. Vecchi. 1983. Optimization by Simulated Annealing. *Science*. 220, 4598 (1983), 671-680.
- [27] R. Agrawal and R. Srikant. 1994. Fast algorithms for mining association rules. In *Proceedings of International Conference on Very Large Data Bases*. 487-499.
- [28] R. Chan R, Q. Yang and Y. D. Shen. 2003. Mining high utility itemsets. In *Proceedings of International Conference on Data Mining*. 19-26.
- [29] W. Song, Y. Liu and J. Li. 2014. BAHUI: Fast and memory efficient mining of high utility itemsets based on bitmap. *Int. J. Data Warehous. Min.*, 10, 1 (2014), 1-15.
- [30] W. Song, Y. Liu and J. Li. 2014. BAHUI: Fast and memory efficient mining of high utility itemsets based on bitmap. *Int. J. Data Warehous. Min.* 10, 1, (2014), 1-15.
- [31] S. Bagui and P. Stanley. 2020. Mining frequent itemsets from streaming transaction data using genetic algorithms. *J. Big Data*. 7, 1 (3030), 54.
- [32] Y. Djenouri, D. Djenouri and A. Belhadi, P. Fournier-Viger and J. C. W. Lin. A new framework for meta heuristic-based frequent itemset mining. *Appl Intell*, 48, 12 (2018), 4775-4791.
- [33] D. Martín, J. Alcalá-Fdez, A. Rosete and F. Herrera. 2016. NICGAR: A niching genetic algorithm to mine a diverse set of interesting quantitative association rules. *Inf Sci*. 355-356, (2016), 208-228.
- [34] E. Alatas and E. Akin. 2006. An efficient genetic algorithm for automated mining of both positive and negative quantitative association rules. *Soft Comput.*, 10, 3, (2006), 230-237.
- [35] J. Alcalá-Fdez, N. F. Pape, A. Bonarini and F. Herrera. 2010 Analysis of the effectiveness of the genetic algorithms based on extraction of association rules. *Fundam. Inform.* 98, 1 (2010), 1-14.
- [36] S. Dehuri, S. Patnaik, A. Ghosh and RR. Mall. 2008. Application of elitist multi-objective genetic algorithm for classification rule generation. *Appl. Soft Comput.* 8, 1 (2008), 477-487.
- [37] P. P. Wakabi-Waiswa, V. Baryamureeba and K. Sarukesi. 2011. Optimized association rule mining with genetic algorithms. In *Proceedings of International Conference on Natural Computation*. 1116-1120.
- [38] X. Yan, X. Zhang and X. Zhang. 2009. Genetic algorithm-based strategy for identifying association rules without specifying actual minimum support. *Expert. Syst. Appl.* 36, 2 (2009), 3066-3076.
- [39] R. Pears and K. S. Koh. 2011. Weighted association rule mining using particle swarm optimization. In *Proceedings of International Workshop on New Frontiers in Applied Data Mining*. 327-338.
- [40] J. Gou, F. Wang and W. Luo. 2015. Mining fuzzy association rules based on parallel particle swarm optimization algorithm. *Intell. Autom. Soft. Comput.* 21, 2 (2015), 147-162.
- [41] Q. Zhang, W. Fang, J. Sun and Q. Wang. 2019. Improved genetic algorithm for high-utility itemset mining. *IEEE Access*. 7 (2019), 176799-176813.
- [42] J. M. T. Wu, J. Zhan and J. C. W. Lin. 2017. An ACO-based approach to mine high-utility itemsets. *Knowl. Based Syst.* 116 (2017), 102-113.
- [43] W. Song and C. Huang. 2020. Mining high average-utility itemsets based on particle swarm optimization. *Data Sci. Patt. Recog.* 4, 2 (2020), 19-32.
- [44] N. Pazhaniraja, S. Sountharajan and B. S. Kumar. High utility itemset mining: A boolean operators-based modified grey wolf optimization algorithm. *Soft Comput.* 24, 21 (2020), 16691-16704.
- [45] H. Yao, H. J. Hamilton and C. J. Butz. 2004. A foundational approach to mining itemset utilities from databases. In *Proceedings of SIAM International Conference on Data Mining*. 482-486.

- [46] H. Yao and H. J. Hamilton. 2006. Mining itemset utilities from transaction databases. *Data Knowl. Eng.* 59, 3 (2006), 603-626.
- [47] M. S. Nawaz, M. Z. Nawaz, O. Hasan, P. Fournier-Viger and M. Sun. An evolutionary/heuristic-based proof searching framework for interactive theorem prover. *Appl. Soft Comput.* 104 (2021), 107200.
- [48] P. Fournier-Viger, J. C. W. Lin, A. Gomariz, T. Gueniche, A. Soltani, Z. Deng and T. H. Lam. 2016. The SPMF open-source data mining library version 2. In *Proceedings of European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases*. 36-40.
- [49] T. Jones. 1995. Crossover, macromutationand, and population-based search. In *Proceedings of International Conference on Genetic Algorithm*. 73-80.
- [50] M. S. Nawaz, P. Fournier-Viger, W. Song, J. C. W. Lin and B. Noack 2021. Investigating crossover operators in genetic algorithms for high-utility itemset mining. In *Proceedings of the Asian Conference on Intelligent Information and Database Systems*. 16-28.
- [51] J. Grobler and A. P. Engelbrecht. 2016. Headless chicken particle swarm optimization algorithms. In *Proceedings of International Conference on Swarm Intelligence*. 350-357.
- [52] T. P. Hong, C. H. Lee and S. L. Wang. 2009. Mining high average-utility itemsets. In *Proceedings of International Conference on Systems, Man, and Cybernetics*. 2526-2530.